Universidad Nacional Autónoma de México

Faculty of Engineering

Electrical Engineering Division

Course: Compilers **Group:** 5

Semester: 2026-2

Report:
Lexical Analyzer (Lexer)

Professor:

M.C. René Adrián Dávila Pérez

Student IDs:

320060829

320317264

320206968

320025561

320266083

Delivery Date:

Sunday, March 15th, 2026

Contents

1 Introduction **2**

2 Theorical Framework **2**
2.1 Lexical Analysis (Scanner / Lexer) 3
2.2 Syntactic analysis (Parsing) and CFG (Context-Free Grammars) 3
2.3 Parsing Strategies: Top-Down Approach and Precedence Resolution 4

3 Developmenmt **4**
3.1 Overview of the implementation 4
3.2 Grammatic 5
3.3 Input/Output and execution 13
3.4 Relation with the propose CFG 13
3.5 Tests realized 14

4 Results **14**
4.1 Input from keyboard 14
4.2 Input from file 16
4.3 Lexical Analysis in Terminal 18
4.4 Grouping of Tokens by Type 18
4.5 Code Visualization with Token Highlighting 18
4.6 Exporting Results 18

5 Conclusions **20**

References **20**

1 Introduction

This document contains the necessary documentation for the understanding in the construction of a lexical analyzer, where it will process source code by reading it from a file or text string. This program will be able to identify, classify and count the tokens in the code that was provided to it, in addition these will be separated into categories such as keywords, identifiers, operators, punctuators, constants and literals.
However, the lexer program meets the restriction of not making use of the FLEX tool, so a lexical analyzer was implemented using the Python Pygments library, which provides a lexer for the C language.
Now then, the lexical analyzer represents the initial phase of a compilation process, where this sets the foundations for the formulation and improvement of a context-free grammar (CFG), this will use a left factorization, which later will be used for the construction of a parser, which will be responsible for validating the syntactic structure of the code that we want to compile, ensuring that it complies with the rules defined in the CFG.

Objectives

- Build a lexical analyzer capable of processing code through the passing of a file or a text string.
- Generate a table with the lexical analysis of the input, classifying the tokens (such as keywords, identifiers, operators, punctuators, constants and literals) and giving a count for each category.
- Provide a basic proposal of Context-Free Grammar (CFG) that represents one of the reading examples, applying left factorization to improve that grammar.
- Package the application following software engineering practices, name it as `unam.fi.compilers.g5.00` and share the repository link with public permissions.

2 Theorical Framework

Firstly, the traduction of a high level programming languaje into a low level one is a process structured in multiple phases. [1]
In this proyect we will aboard the inicial stages of the front-end of the compiler: the lexical analysis and the syntatic analysis. This phases work together in order to validate that the source code given by the user is well formed before proceding with the semantic evaluation or the code generation. In this case, the validation procees its aplied to a code that is written in C that uyses a specific set of instructions

2.1 Lexical Analysis (Scanner / Lexer)

The lexical analysis represents the first stage of the languaje processing. Its fundamental responsability is to redd the code character by character and group them in secuences with logical meaning, known as lexemes.[1]
Starting from this lexemes, the analyzer, generatees a secuence of tokens (as identifiers, key words, constants, special characters, operators, punctuation, literals). In the proyect, the lexical analyzer identifies specific sets as:

- **ID:** Identifiers used to give name to the variables
- **VALUE_NUM:** Literals that represent any number whole and decimal
- **VALUE_CHAR:** Literals that represent a unique character enclosed between simple commas
- **TEXT_STRING:** Text strings enclosed in double quotes, mostly used as format specifiers

2.2 Syntactic analysis (Parsing) and CFG (Context-Free Grammars)

The syntatic analyzer is the second stage in this compiler. It recibes the flow of tokens generate by the lexical analizer and verifies that the secuence fullfill the structural rules of the language. [1]
To define this rules in a rigurous way, it uses the CFG (CONTEXT-FREE GRAMMARS), it helps to stablish which rules are valid using the production rules.[1]
In this proyect, the CFG defines the structure to prodecue any instruction (StmtList), covering numeric declarations (DeclNum), assignments (AssignSmt), and flow control statements (lfStmt, WhileStmt, ForStmtm)
An example given is the formal rule of declaring a number (and optionally initialize it) it defines matematically in the next way:

$$DeclNum- > TypeNumIDAssignOpNum$$

The main objective of the parser is to organize the sequence of tokens into a hierarchical structure called a parse tree (or syntax tree). This structure shows how the tokens relate to each other according to the language rules and confirms that the code follows the correct syntax [2]

2.3 Parsing Strategies: Top-Down Approach and Precedence Resolution

There are two main ways to construct the syntax tree: Top-Down and Bottom-Up. This project implements a Top-Down approach through a Recursive Descent Parser. In this method, the tree is built starting from the initial symbol of the grammar down to the leaves (the tokens), associating a recursive function with each grammar rule to process the input from left to right. [2]
To ensure that this approach works mathematically without entering infinite loops and respecting the operation hierarchies, the grammar was designed with specific features:

- **Elimination of left recursion (Left-Recursion):** The production rules for mathematical expressions (MathExpr) were rewritten to be right-recursive.
- **Layered Precedence:** To evaluate logical conditions (Cond) while respecting the standardized hierarchy, the rules were divided into levels. The analysis starts at the lowest priority level looking for the OR logical operator (CondOrPrime), descends to the intermediate AND level (CondAndPrime), evaluates the level of relational comparisons (CondRel), and culminates at the highest priority level processing the logical NOT and parentheses (CondUnary).

3 Developmenmt

3.1 Overview of the implementation

The implementation of the lexer is written in Python, but our CFG considers code write in C, with the instructions describe in SmtList and we get support in pygments of lex in order to analize the input lines, either by keyboard or a file.
At the moment of using Pygments, we took in account that the family of the Pygments tokens do not exactly coincides with the tokens seeing in course, due to the asociation will stay the next way:

Family of Tokens (Pygments)	Course Category
Token.Keyword	KEYWORD
Token.Name	IDENTIFIER
Token.Operator	OPERATOR
Token.Punctuation	PUNCTUATION
Token.Literal.Number	CONSTANT
Token.Literal / Token.Literal.String	LITERAL

Table 1: Mapping of Pygments tokens to the course categories.

It should be noted that the special cases of string literals delimited by quotes on the same line were considered, grouping the opening quote, the content, and the closing quote into a single token so that it is recognized as a literal.
At the end of the program, after processing the input it must print a table with the lexical analysis of the input and the count of set of tokens

3.2 Grammatic

1. Production Rules

$$
\begin{aligned}
\text{Stmt} &\rightarrow \text{Stmt StmtList} \mid \varepsilon \\
\text{StmtList} &\rightarrow \text{DeclNum} \mid \text{DeclChar} \mid \text{AssignStmt} \mid \text{IfStmt} \mid \text{WhileStmt} \\
&\quad \mid \text{ForStmt} \mid \text{PrintStmt} \mid \text{ScanStmt} \mid \text{UnaryStmt}
\end{aligned}
$$

Donde:

- **Stmt** Is the initial symbol to generate instructions.
- **StmtList** Represents the specific instruction that can be produced.
- **DeclNum** y **DeclChar** Its the declarations of numbers and characters.
- **AssignStmt** Is for the variables asignation.
- **IfStmt, WhileStmt, ForStmt** they are the flow control structures.
- **PrintStmt, ScanStmt** they are the input and output functions of data.
- **UnaryStmt** it is for unary statements (pre/post increments).

2. Production rules:

$$
\begin{aligned}
\text{DeclNum} &\rightarrow \text{TypeNum ID AssignOpNum;} \\
\text{AssignOpNum} &\rightarrow \text{MathExpr} \mid \varepsilon \\
\text{DeclChar} &\rightarrow \text{TypeChar ID AssignOpChar;} \\
\text{AssignOpChar} &\rightarrow \text{VALUE_CHAR} \mid \varepsilon \\
\text{AssignStmt} &\rightarrow \text{ID AssignExprOp MathExpr} \mid \text{ID VALUE_CHAR;} \\
\text{TypeNum} &\rightarrow \text{int} \mid \text{float} \mid \text{double} \\
\text{TypeChar} &\rightarrow \text{char}
\end{aligned}
$$

Donde:

- DeclNum : Is the PR to declare a number or declare a number and initialize it.
- DeclChar : PR to declare a character or declare a character with a number and initialize it.
- AssignStmt : Is the PR for assigning a value to a data previously declared.
- TypeNum : The data type numeric that it accepts.
- TypeChar : Tipe of character data that accepts.

3. Applying left factoring:
The variable declarations $DeclNum \rightarrow TypeNum\ ID\ ; \mid TypeNum\ ID\ =\ MathExpr\ ;$ has the common factor $TypeNum\ ID$, so we can express it like:

$$\begin{aligned} \text{DeclNum} &\rightarrow \text{TypeNum ID AssignOpNum;} \\ \text{AssignOpNum} &\rightarrow \text{MathExpr} \mid \varepsilon \end{aligned}$$

Also the expression for continuous operations

$$\text{MathExpr} \rightarrow \text{MathExpr MathOp MathOperand} \mid \text{MathOperand}$$

presents left recursion, so it was factored (right recursion) as follows:

$$\begin{aligned} \text{MathExpr} \rightarrow\ & \text{MathOperand} \\ & \mid \text{MathOperand MathOp MathExpr} \end{aligned}$$

This procedure will help to avoid non-determinism, also with the help of the right recursion already done, where we have non-terminal elements on the right side, so this can help with the repetitive and recursive strings that we can make when nesting operations or logical calls. Finally the gramatic will be defined by the expressions:

$$
\begin{aligned}
\text{Stmt} &\rightarrow \text{Stmt StmtList} \mid \varepsilon \\
\text{StmtList} &\rightarrow \text{DeclNum} \mid \text{DeclChar} \mid \text{AssignStmt} \mid \text{IfStmt} \mid \text{WhileStmt} \mid \\
&\quad \text{ForStmt} \mid \text{PrintStmt} \mid \text{ScanStmt} \mid \text{UnaryStmt} \\
\text{DeclNum} &\rightarrow \text{TypeNum ID AssignOpNum;} \\
\text{AssignOpNum} &\rightarrow \text{MathExpr} \mid \varepsilon \\
\text{DeclChar} &\rightarrow \text{TypeChar ID AssignOpChar;} \\
\text{AssignOpChar} &\rightarrow \text{VALUE_CHAR} \mid \varepsilon \\
\text{AssignStmt} &\rightarrow \text{ID AssignExprOp MathExpr} \mid \text{ID VALUE_CHAR;} \\
\text{TypeNum} &\rightarrow \text{int} \mid \text{float} \mid \text{double} \\
\text{TypeChar} &\rightarrow \text{char} \\
\text{IfStmt} &\rightarrow \text{if (Cond) \{StmtList\}} \\
\text{WhileStmt} &\rightarrow \text{while (Cond) \{StmtList\}} \\
\text{ForStmt} &\rightarrow \text{for (ForInit; ForCond; ForStep) \{StmtList\}} \\
\text{ForInit} &\rightarrow \text{TypeNum ID AssignOpNum} \mid \text{ID AssignExprOp MathExpr} \mid \varepsilon \\
\text{ForCond} &\rightarrow \text{Cond} \mid \varepsilon \\
\text{ForStep} &\rightarrow \text{IncDecOp ID} \mid \text{ID IncDecOp} \mid \\
&\quad \text{ID AssignExprOp MathExpr} \mid \varepsilon \\
\text{PrintStmt} &\rightarrow \text{printf(PrintArgs);} \\
\text{PrintArgs} &\rightarrow \text{TEXT_STRING} \mid \text{TEXT_STRING, IdList;} \\
\text{ScanStmt} &\rightarrow \text{scanf(TEXT_STRING, \&ID);} \\
\text{IdList} &\rightarrow \text{ID} \mid \text{ID, IdList} \\
\text{UnaryStmt} &\rightarrow \text{IncDecOp ID;} \mid \text{ID IncDecOp;} \\
\text{IncDecOp} &\rightarrow ++ \mid - \\
\text{AssignExprOp} &\rightarrow = \mid += \mid \text{-=} \mid /= \\
\text{MathExpr} &\rightarrow \text{MathOperand} \mid \text{MathOperand MathOp MathExpr} \\
\text{MathOperand} &\rightarrow \text{ID} \mid \text{VALUE_NUM} \\
\text{MathOp} &\rightarrow + \mid - \mid * \mid / \mid \% \\
\text{Cond} &\rightarrow \text{CondAnd CondOrPrime} \\
\text{CondOrPrime} &\rightarrow || \text{ CondAnd CondOrPrime} \mid \varepsilon \\
\text{CondAnd} &\rightarrow \text{CondRel CondAndPrime} \\
\text{CondAndPrime} &\rightarrow \&\& \text{ CondRel CondAndPrime} \mid \varepsilon \\
\text{CondRel} &\rightarrow \text{MathExpr RelOp MathExpr} \mid \text{CondUnary} \\
\text{CondUnary} &\rightarrow ! \text{ CondUnary} \mid \text{(Cond)} \mid \text{IdVal} \\
\text{IdVal} &\rightarrow \text{ID} \mid \text{VALUE_NUM} \mid \text{VALUE_CHAR} \\
\text{RelOp} &\rightarrow == \mid ! = \mid < \mid > \mid <= \mid >=
\end{aligned}
$$

4. Logical conditions (Layer precedence)
For rigorously respecting the hierarchy of logical operations, the production rules for conditions (`Cond`) were designed using a stratified priority level approach:

- **Nivel de Menor Prioridad (OR):** The analysis starts in `Cond`, obligating the evaluation of lower layers before processing the `||` operator.
- **Nivel Intermedio (AND):** Through `CondAnd`, the operators group their logical operands before the upper layer (OR) can process them.
- **Nivel de Comparaciones (Relacional):** The rule `CondRel` evaluates the mathematical expressions separated by relational operators ($>$, $<$, $==$).
- **Nivel de Máxima Prioridad (NOT y Paréntesis):** The rule `CondUnary` ensures that logical negations and nested expressions between parentheses are resolved first.

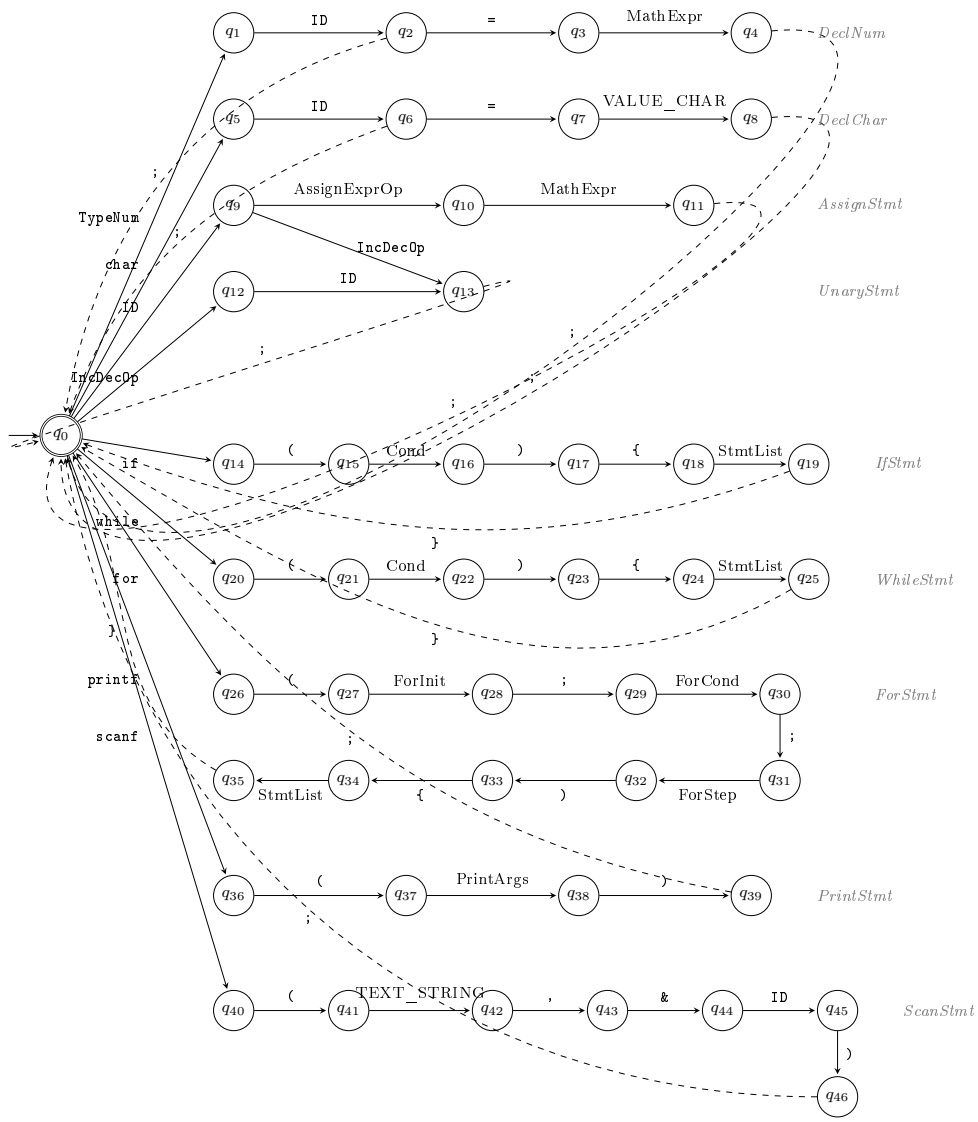
5.Deterministic Finite Automaton (DFA) of the Predictive Syntax Analyzer

It presents the deterministic finite automaton that models the behavior of the predictive top-down syntax analyzer based on the context-free grammar defined above. The state q_0 is the initial and accepting state (representing the production Stmt $\rightarrow \varepsilon$). Each path from q_0 corresponds to the recognition of a type of instruction; upon completion, the automaton returns to q_0 to process the next instruction.

Table of Transitions for production:

Production	Source State	Symbol	Destination State
DeclNum	q_0	TypeNum	q_1
	q_1	ID	q_2
	q_2	`=`	q_3
	q_3	MathExpr	q_4
	q_4	`;`	q_0
	q_2	`;` (ε)	q_0
DeclChar	q_0	`char`	q_5
	q_5	ID	q_6
	q_6	`=`	q_7
	q_7	VALUE_CHAR	q_8
	q_8	`;`	q_0
	q_6	`;` (ε)	q_0
AssignStmt	q_0	ID	q_9
	q_9	AssignExprOp	q_{10}
	q_{10}	MathExpr	q_{11}
	q_{11}	`;`	q_0
UnaryStmt (prefijo)	q_0	IncDecOp	q_{12}
	q_{12}	ID	q_{13}
	q_{13}	`;`	q_0
UnaryStmt (posfijo)	q_9	IncDecOp	q_{13}
	q_{13}	`;`	q_0
IfStmt	q_0	`if`	q_{14}
	q_{14}	`(`	q_{15}
	q_{15}	Cond	q_{16}
	q_{16}	`)`	q_{17}
	q_{17}	`{`	q_{18}
	q_{18}	StmtList	q_{19}
	q_{19}	`}`	q_0

Production	Source State	Symbol	Destination State
WhileStmt	q_0	`while`	q_{20}
	q_{20}	`(`	q_{21}
	q_{21}	Cond	q_{22}
	q_{22}	`)`	q_{23}
	q_{23}	`{`	q_{24}
	q_{24}	StmtList	q_{25}
	q_{25}	`}`	q_0
ForStmt	q_0	`for`	q_{26}
	q_{26}	`(`	q_{27}
	q_{27}	ForInit	q_{28}
	q_{28}	`;`	q_{29}
	q_{29}	ForCond	q_{30}
	q_{30}	`;`	q_{31}
	q_{31}	ForStep	q_{32}
	q_{32}	`)`	q_{33}
	q_{33}	`{`	q_{34}
	q_{34}	StmtList	q_{35}
	q_{35}	`}`	q_0
PrintStmt	q_0	`printf`	q_{36}
	q_{36}	`(`	q_{37}
	q_{37}	PrintArgs	q_{38}
	q_{38}	`)`	q_{39}
	q_{39}	`;`	q_0
ScanStmt	q_0	`scanf`	q_{40}
	q_{40}	`(`	q_{41}
	q_{41}	TEXT_STRING	q_{42}
	q_{42}	`,`	q_{43}
	q_{43}	`&`	q_{44}
	q_{44}	ID	q_{45}
	q_{45}	`)`	q_{46}
	q_{46}	`;`	q_0



Leyenda: The point arrows (- - -) indicate the return to stateq_0.
The labels in *grey cursive* identify the corresponding production.

Figure 1: DFA del analizador sintáctico predictivo para la gramática propuesta.

Formal Description of the Automaton:

The deterministic finite automaton is formally defined as the 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where:

- $Q = \{q_0, q_1, q_2, \ldots, q_{46}\}$ (47 states)
- Σ is the set of input symbols (terminals and non-terminals of the grammar)

- $\delta : Q \times \Sigma \to Q$ is the transition function defined below
- q_0 is the initial state
- $F = \{q_0\}$ is the set of accepting states

Observations about the automaton:

1. **State q_0 as initial and accepting state:** The state q_0 fulfills both roles because the production Stmt $\to \varepsilon$ allows accepting the empty string (a program without statements). After completely recognizing a statement, the automaton always returns to q_0 to initiate the recognition of the next one.
2. **Determinism by anticipation token (*lookahead*):** From q_0, the first token of the input determines unequivocally the path to follow:
   - `int`, `float`, `double` $\to$ DeclNum (state q_1)
   - `char` $\to$ DeclChar (state q_5)
   - `ID` $\to$ AssignStmt o UnaryStmt posfijo (state q_9)
   - `++`, `--` $\to$ UnaryStmt prefijo (state q_{12})
   - `if` $\to$ IfStmt (state q_{14})
   - `while` $\to$ WhileStmt (state q_{20})
   - `for` $\to$ ForStmt (state q_{26})
   - `printf` $\to$ PrintStmt (state q_{36})
   - `scanf` $\to$ ScanStmt (state q_{40})
3. **Optional transitions(ε):** The states q_2 y q_6 posses two transitions for `;`: uone direct to q_0 (when the variable is declared without initialization, it means AssignOpNum $\to \varepsilon$) and another that passes through the assignment before reaching the final `;`.
4. **Ambiguety reolved in q_9:** When the first token is `ID`, the automaton transitions to q_9. From there, the second token resolves the ambiguity:
   - If it is an *AssignExprOp* (`=`, `+=`, `-=`, `/=`) $\to$ the path of AssignStmt is followed.
   - If it is an *IncDecOp* (`++`, `--`) $\to$ the path of UnaryStmt posfijo is followed.
5. **Non-terminal symbols as transitions:** The labels like *Cond*, *StmtList*, *MathExpr*, *PrintArgs*, *ForInit*, *ForCond* y *ForStep* represent recursive calls to the predictive parser, where each non-terminal expands according to its own production rules defined in the grammar.

3.3 Input/Output and execution

The lexical analyzer developed supports two modes of operation for C language inputs: reading from a file and reading from the keyboard.

1. **Read from file.** In this mode, the user must place the file in the inputs folder and the system reads it using UTF-8 encoding and subsequently processes the content line by line. At the end of the lexical analysis, the results can be exported to the outputs folder.

2. **Read from keyboard.** The analyzer allows entering source code directly from the terminal. The capture ends when the user inputs the terminal symbol `$`, which acts as a delimiter for the end of input.

During execution, the system performs the following main operations:

- Tokenization of the source code using the C lexer provided by the *Pygments* library.
- Classification of each token according to the categories defined in the course (Keyword, Identifier, Operator, Punctuator, Constant and Literal).
- Removal of irrelevant tokens such as whitespace or line breaks that do not provide syntactic information.
- Grouping and total counting of tokens by category.
- Optional visualization of the source code with token highlighting in colors and exportation of results to a text file.

It is worth mentioning that the particular case of the character `&` used in the `scanf()` function, as well as certain delimiters such as parentheses, braces, commas and semicolons were explicitly treated as punctuators because the library used does not categorize them consistently.

3.4 Relation with the propose CFG

The design of the lexical analyzer was carried out taking as reference the context-free grammar defined for the reduced C language considered in this project. Our CFG includes instructions such as declarations of numeric and character variables, assignments, control structures (`if`, `while`, `for`), unary increment and decrement operations, as well as calls to input/output functions (`printf` and `scanf`) **cfglexer**.
Also we defined production rules for mathematical expressions and logical conditions respecting operator precedence through a layered derivation approach, where logical OR operators have the lowest priority, followed by AND, relational operators and finally unary operators such as logical negation and the use of parentheses.

3.5 Tests realized

In order to validate our lexical analyzer, a couple of tests were proposed using fragments of code in the C language that include declarations, arithmetic expressions, calls to input/output functions and control structures contemplated in the proposed CFG.

- **Code 1: Statements and arithmetic expressions**

  In this test, a program was analyzed that includes declarations of numeric variables, an assignment with an arithmetic expression and a call to the `printf()` function. The lexical analyzer correctly identified keywords, identifiers, assignment operators and arithmetic operators, as well as numeric constants and language delimiters.

  Similarly, the proper recognition of the string literal used in the output function was verified, which was grouped as a single token. The result of the analysis made it possible to obtain the total count of tokens classified by category, validating the correct tokenization of arithmetic expressions and print statements.

- **Code 2: Data input and iterative structure**

  In this test, a program was evaluated that performs data reading using `scanf()` and employs an iterative structure `for`. During the analysis, the associated keywords for the loop, relational and increment operators were correctly recognized, as well as the corresponding delimiters.

  Additionally, the classification of the special symbol `&` used for reference reading and the correct identification of the string literal within the `printf()` function were verified. The results confirmed that the lexical analyzer can adequately process iterative statements and calls to input/output functions, generating the table and grouped count of tokens.

4 Results

To validate the functioning of the lexical analyzer, two program execution tests were carried out, one using keyboard input and another using input from a file. In both cases, the system's behavior during the analysis of the source code can be observed through the terminal, as well as the generation of the corresponding output file.

4.1 Input from keyboard

To verify the operation of the lexical analyzer through keyboard input, we execute the program main.py without arguments from the terminal. The system then asks us to enter code in the C language line by line, and the input capture ends when the symbol $ is entered. For this case, we used the same code contained in the file code.c, entered manually through the keyboard.

```
(.venv) PS C:\Users\rodyl\OneDrive\Escritorio\FacultadIngenieria\8vo_Semestre\Compiladores\LEXER\Compiladores_Documentacion\python> python src./main.py

========================================
          Keyboard Input
========================================

Write your source code in C language
NOTE: When you are done, type '$' to finish the input.

int main() {
    int x, a = 2, b = 3, c = 5;
    x = a + b * c;

    printf("The value of x is: %d", x);
    return 0;
}
$

==================================================
          Lexical Analysis of <keyboard_input>

==================================================

==================================================
TOKEN TYPE      | LEXEMS
==================================================
KEYWORD         | int
IDENTIFIER      | main
PUNCTUATION     | (
PUNCTUATION     | )
PUNCTUATION     | {
KEYWORD         | int
IDENTIFIER      | x
PUNCTUATION     | ,
IDENTIFIER      | a
OPERATOR        | =
CONSTANT        | 2
PUNCTUATION     | ,
IDENTIFIER      | b
OPERATOR        | =
CONSTANT        | 3
PUNCTUATION     | ,
IDENTIFIER      | c
OPERATOR        | =
CONSTANT        | 5
```

Figure 2: Terminal – Start of the program, capture of source code via keyboard, and results of the lexical analysis.

```
CONSTANT        | 5
PUNCTUATION     | ;
IDENTIFIER      | x
OPERATOR        | =
IDENTIFIER      | a
OPERATOR        | +
IDENTIFIER      | b
OPERATOR        | *
IDENTIFIER      | c
PUNCTUATION     | ;
IDENTIFIER      | printf
PUNCTUATION     | (
LITERAL         | "The value of x is: %d"
PUNCTUATION     | ,
IDENTIFIER      | x
PUNCTUATION     | )
PUNCTUATION     | ;
KEYWORD         | return
CONSTANT        | 0
PUNCTUATION     | ;
PUNCTUATION     | }
==================================================
Total tokens found: 39

> Do you want to see the grouped tokens by type? (y/n): Y

 GROUPED TOKENS BY TYPE:

KEYWORD (Total = 3):
        int: 2 times
        return: 1 times
IDENTIFIER (Total = 11):
        main: 1 times
        x: 3 times
        a: 2 times
        b: 2 times
        c: 2 times
        printf: 1 times
```

Figure 3: Terminal – Grouping of tokens by type.

Figure 4: Terminal – Grouping of tokens by type and visualization of the source code with token highlighting.

4.2 Input from file

To validate the program's functionality with source files, we carried out a similar process, but this time using the file code1.c stored in the inputs directory. The program was executed by specifying the file name as an argument in the command line.

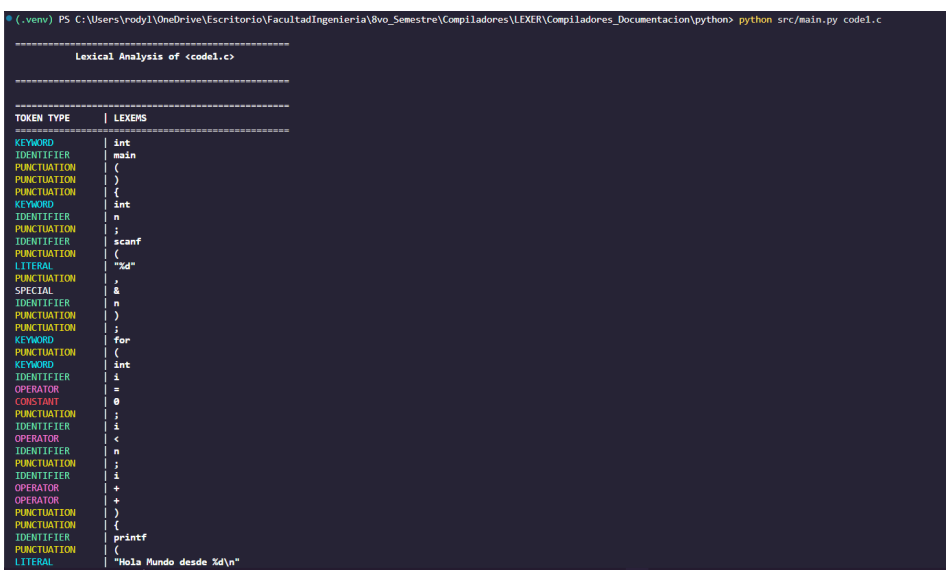


Figure 5: Terminal – Start of the program, capture of source code via keyboard, and results of the lexical analysis.

```
(.venv) PS C:\Users\rodyl\OneDrive\Escritorio\FacultadIngenieria\8vo_Semestre\Compiladores\LEXER\Compiladores_Documentacion\python> python src/main.py code1.c
LITERAL         | "Hola Mundo desde %d\n"
PUNCTUATION     | ,
IDENTIFIER      | i
PUNCTUATION     | )
PUNCTUATION     | ;
PUNCTUATION     | }
KEYWORD         | return
CONSTANT        | 0
PUNCTUATION     | ;
PUNCTUATION     | }
==================================================
Total tokens found: 44

> Do you want to see the grouped tokens by type? (y/n): Y

 GROUPED TOKENS BY TYPE:

KEYWORD (Total = 5):
        int: 3 times
        for: 1 times
        return: 1 times
IDENTIFIER (Total = 10):
        main: 1 times
        n: 3 times
        scanf: 1 times
        i: 4 times
        printf: 1 times
PUNCTUATION (Total = 20):
        (: 4 times
        ): 4 times
        {: 2 times
        ;: 6 times
        ,: 2 times
        }: 2 times
OPERATOR (Total = 4):
        =: 1 times
        <: 1 times
        +: 2 times
CONSTANT (Total = 2):
        0: 2 times
LITERAL (Total = 2):
        "%d": 1 times
        "Hola Mundo desde %d\n": 1 times
```

Figure 6: Terminal – Grouping of tokens by type.

```
(.venv) PS C:\Users\rodyl\OneDrive\Escritorio\FacultadIngenieria\8vo_Semestre\Compiladores\LEXER\Compiladores_Documentacion\python> python src/main.py code1.c
        (: 4 times
        ): 4 times
        {: 2 times
        ;: 6 times
        ,: 2 times
        }: 2 times
OPERATOR (Total = 4):
        =: 1 times
        <: 1 times
        +: 2 times
CONSTANT (Total = 2):
        0: 2 times
LITERAL (Total = 2):
        "%d": 1 times
        "Hola Mundo desde %d\n": 1 times
SPECIAL (Total = 1):
        &: 1 times

 Total all tokens: 44

> Do you want to see  the original code with the tokens highlighted? (y/n): Y

=== Source Code with the tokens highlighted ===

Color Legend:
  [ KEYWORD ]  [ IDENTIFIER ]  [ PUNCTUATION ]  [ OPERATOR ]  [ CONSTANT ]  [ LITERAL ]  [ SPECIAL ]
--------------------------------------------------------------------------------------------------
int main(){
    int n;
    scanf("%d", &n);

    for(int i = 0; i < n; i++){
        printf("Hola Mundo desde %d\n", i);
    }

    return 0;
}

========================================
> Do you want to export the results? (y/n): Y
Results exported successfully to: C:\Users\rodyl\OneDrive\Escritorio\FacultadIngenieria\8vo_Semestre\Compiladores\LEXER\Compiladores_Documentacion\python\outputs\code1_output.txt

Would you like to run a new analysis? (y/n): N
(.venv) PS C:\Users\rodyl\OneDrive\Escritorio\FacultadIngenieria\8vo_Semestre\Compiladores\LEXER\Compiladores_Documentacion\python>
```

Figure 7: Terminal – Grouping of tokens by type and visualization of the source code with token highlighting.

4.3 Lexical Analysis in Terminal

In the screenshots of both obtained results, we observe how once the code is entered either through the keyboard or from a file, we can see how the program performs the lexical analysis and shows us in the terminal the detected tokens along with their classification.

Each token is highlighted with a specific color to facilitate its identification:

- **Keywords & Literals:** Shown in blue
- **Identifiers:** Shown in green
- **Punctuations:** Shown in yellow
- **Operators:** Shown in pink
- **Constants:** Shown in red
- **Specials:** Shown in white

4.4 Grouping of Tokens by Type

Afterwards, the program asks us if we want to group the tokens by type. If we accept, the system reorganizes the output by showing the tokens classified by category, grouping all the elements that belong to the same lexical type and how many times each one appears. This makes it easier for us to observe the distribution of tokens found within the source code.

4.5 Code Visualization with Token Highlighting

The program also allows us to visualize again the original source code with the tokens highlighted according to their lexical type.
This functionality allows us to compare the original code with the classification performed by the analyzer, clearly showing how each part of the program is identified.

4.6 Exporting Results

Near the end, the program asks us as users if we want to export the results of the analysis, which will be saved in a text file inside the outputs directory. Finally, we decide whether we want to perform a new analysis or finish the execution of the program.

A continuation, screenshots of the output file generated in both cases are shown, where the same format of token classification and grouping that is displayed in the terminal can be observed, but now stored in a text document for later consultation or analysis.

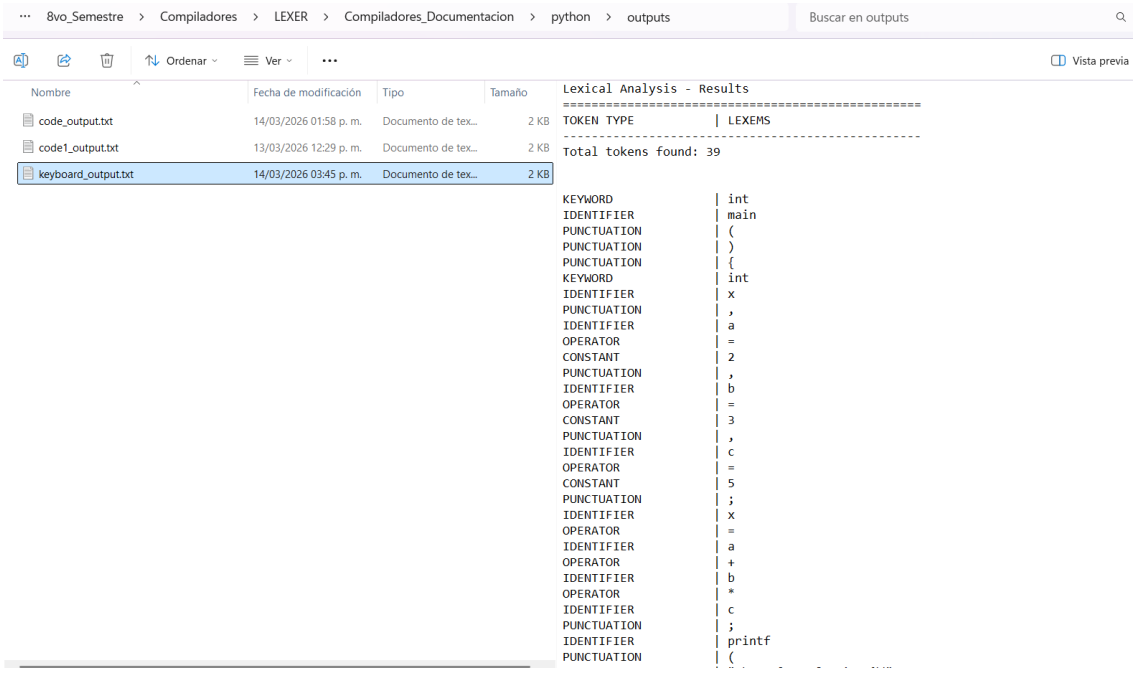


Figure 8: Output file – Results of the analysis from keyboard input.

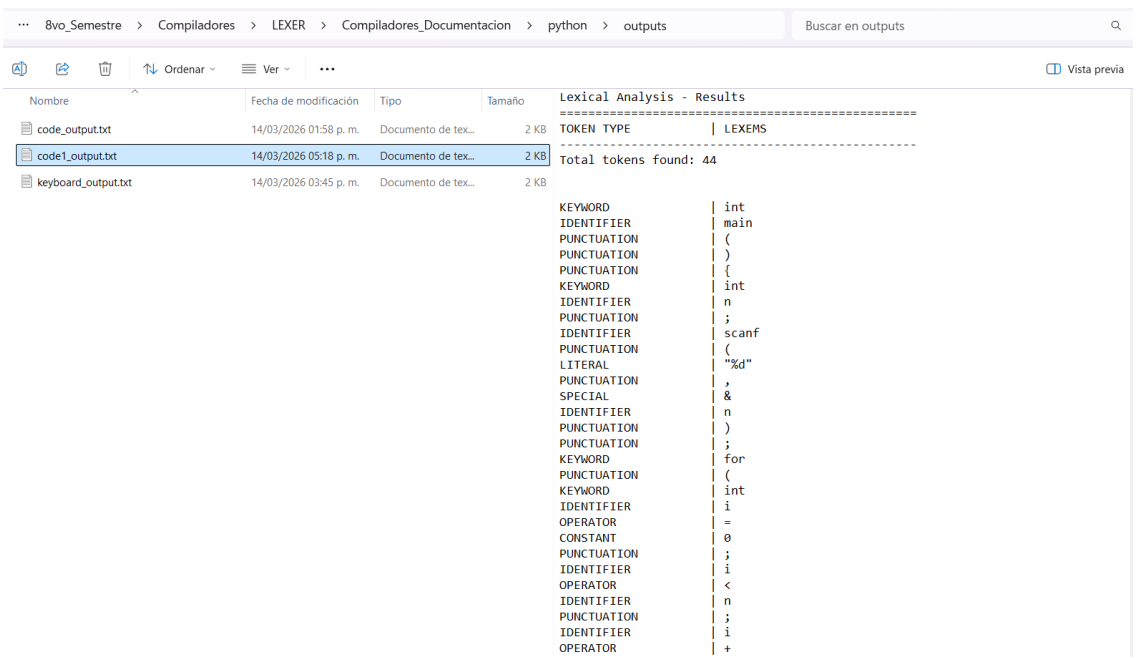


Figure 9: Output file – Results of the analysis from file input.

5 Conclusions

In this work, based on the study of how a compiler functions and the relationship between its different phases, we implemented a lexical analyzer capable of processing fragments of code in the C language and identifying the lexical elements present in the program, which establishes the foundation for the subsequent stages.

During the development, we established a correspondence between the token categories used in the course and the token families provided by the Pygments library, maintaining the classification of Keywords, Identifiers, Punctuation Signs, Operators, Constants, and Special Characters. This allowed the analyzer to correctly identify the lexemes of the source code and organize them into classified tokens. In addition, the functionalities for grouping tokens, visualizing the code with highlighting, and exporting results allow us to observe more clearly the relationship between the source code and its lexical structure.

Similarly, the design of the analyzer was carried out considering the context-free grammar proposed for a subset of the C language, which allows establishing a direct connection between lexical analysis and the subsequent syntactic analysis. This relationship is fundamental within the architecture of a compiler, since the tokens generated by the lexer constitute the input that the parser will use to verify the grammatical structure of the program.

References

[1] A. Holub, *Compiler Design in C*. Prentice-Hall, 1990, ISBN: 0-13-155045-4. [Online]. Available: https://holub.com/goodies/compiler/compilerDesignInC.pdf.

[2] Pygments Contributors. "Builtin tokens." (2026), [Online]. Available: https://pygments.org/docs/tokens/ (visited on 03/13/2026).

[3] G. "What is contextfree grammar?" (Jul. 23, 2025), [Online]. Available: https://www.geeksforgeeks.org/theory-of-computation/what-is-context-free-grammar/.

[4] G. "Ambiguous grammar." (Feb. 19, 2026), [Online]. Available: https://www.geeksforgeeks.org/compiler-design/ambiguous-grammar/.

[5] C. Wong. "Cfg developer." (2014), [Online]. Available: https://web.stanford.edu/class/archive/cs/cs103/cs103.1156/tools/cfg/.